



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



تکلیف شماره ۵

۱ سوال (۲)

نادرست است.

در BGP وقتی یک روتر «اعلان مسیر» را از یک همسایه دریافت می کند، الزاماً نه آن را به «تمام همسایه ها» می فرستد و نه همیشه چیزی به نام «هویت روتر» را به مسیر اضافه می کند. رفتار BGP سیاست محور (Policy-based) و وابسته به نوع همسایگی (iBGP/eBGP) است.

۱. اضافه شدن «هویت» در BGP دقیقاً چیست؟ BGP هنگام اعلان مسیر به بیرون AS، شماره AS خود را به ویژگی AS-PATH اضافه می کند (نه «شناسه روتر»). بنابراین حتی از این جهت هم جمله دقیق نیست: چیزی که افزوده می شود «AS Number» است، نه هویت روتر.

۲. ارسال به تمام همسایه ها درست نیست (سیاست های Import/Export): بعد از دریافت مسیر، روتر ابتدا طبق Import Policy تصمیم می گیرد مسیر را بپذیرد یا فیلتر کند. حتی اگر مسیر پذیرفته شود، طبق Export Policy ممکن است فقط به برخی همسایه ها اعلان شود (مثلاً جلوگیری از Transit Traffic بین دو ISP از داخل یک AS).

۳. فقط مسیر منتخب اعلان می شود (قانون Best Path): برای یک پیشوند ممکن است چند مسیر از همسایه های مختلف برسد؛ روتر معمولاً یک مسیر برتر را انتخاب می کند و (در صورت مجاز بودن سیاست خروجی) همان را اعلان می کند، نه همه مسیرهای دریافتی را.

۴. قانون iBGP (جلوگیری از حلقه): اگر مسیری از طریق iBGP یاد گرفته شود، طبق قاعده iBGP split-horizon روتر اجازه ندارد آن را به یک همسایه iBGP دیگر بازنشر کند (مگر با Route Reflector یا Confederation). پس «ارسال به تمام همسایه ها» درون AS به طور کلی برقرار نیست.

۵. اعلان به همان همسایه فرستنده هم الزاماً انجام نمی شود: به طور معمول مسیر به همسایه ای که همان مسیر را از او یاد گرفته ایم، دوباره اعلان نمی شود.

صورت علمی درست تر:

«روتر BGP پس از اعمال Import Policy و انتخاب Best Path، در صورت مجاز بودن Export Policy،

اگر به همسایه eBGP اعلان کند، شماره AS خود را به AS-PATH اضافه کرده و فقط به همسایه های مجاز اعلان می کند.»

۲ سوال ۴

با توجه به شکل (گره های U, V, W, X, Y و هزینه لینک ها)، داریم:

۱. بردار فاصله نهایی مسیریاب Y (پس از همگرایی)

پس از همگرایی، هر مقدار برابر کم هزینه ترین مسیر است:

$$d_Y(Y) = ۰, \quad d_Y(X) = ۶,$$

$$d_Y(W) = ۶ + ۴ = ۱۰ \quad (Y \rightarrow X \rightarrow W),$$

$$d_Y(V) = ۶ + ۶ = ۱۲ \quad (Y \rightarrow X \rightarrow V),$$

$$d_Y(U) = ۶ + ۶ + ۲ = ۱۴ \quad (Y \rightarrow X \rightarrow V \rightarrow U).$$

بنابراین با ترتیب خواسته شده (y, x, w, v, u) :

$$(y, x, w, v, u) = (۰, ۶, ۱۰, ۱۲, ۱۴).$$

۲. بردار فاصله اولیه برای مسیریاب U (در $t = ۰$)

در ابتدا هر روتر فقط همسایه مستقیم خود را می شناسد؛ U فقط به V با هزینه ۲ وصل است و بقیه ∞ هستند (طبق صورت سؤال، ∞ را با x می نویسیم):

$$(y, x, w, v, u) = (x, x, x, ۲, ۰).$$

۳. مشکل هنگام افزایش هزینه لینک

در الگوریتم Distance Vector «خبر خوب سریع پخش می شود»، اما هنگام افزایش هزینه لینک (یا قطع لینک)، ممکن است روترها به صورت تدریجی و اشتباه هزینه ها را زیاد کنند و در حلقه های موقت گیر بیفتند. نام این پدیده مشکل شمارش تا بی نهایت (Count-to-Infinity) است (که با Routing Loop موقت همراه می شود).

۳ سوال ۶

در شکل، هزینه‌ها داریم:

$$c(x, w) = ۲, \quad c(w, y) = ۲, \quad c(x, y) = ۵,$$

و نیز:

$$D_w(u) = ۵, \quad D_y(u) = ۶.$$

همچنین چون w و y مستقیماً به هم وصل‌اند:

$$D_w(y) = ۲, \quad D_y(w) = ۲.$$

در الگوریتم Distance Vector (بلمن-فورد):

$$D_x(d) = \min_{v \in N(x)} \{c(x, v) + D_v(d)\}, \quad N(x) = \{w, y\}.$$

(الف) بردار فاصله x برای مقاصد w, y, u

$$D_x(w) = \min\{c(x, w), c(x, y) + D_y(w)\} = \min\{۲, ۵ + ۲\} = ۲.$$

$$D_x(y) = \min\{c(x, y), c(x, w) + D_w(y)\} = \min\{۵, ۲ + ۲\} = ۴.$$

$$D_x(u) = \min\{c(x, w) + D_w(u), c(x, y) + D_y(u)\} = \min\{۲ + ۵, ۵ + ۶\} = \min\{۷, ۱۱\} = ۷.$$

پس:

$$(D_x(w), D_x(y), D_x(u)) = (۲, ۴, ۷).$$

(ب) تغییر هزینه لینک به گونه‌ای که x مسیر کمینه جدید به u را اعلام کند

کافی است یکی از هزینه‌های $c(x, w)$ یا $c(x, y)$ را طوری تغییر دهیم که مقدار $D_x(u)$ عوض شود. مثلاً اگر

$$c(x, w) : ۲ \rightarrow ۱,$$

آنگاه

$$D_x(u) = \min\{۱ + ۵, ۵ + ۶\} = \min\{۶, ۱۱\} = ۶,$$

بنابراین مقدار $D_x(u)$ از ۷ به ۶ تغییر می‌کند و در نتیجه x یک update جدید به همسایه‌هایش می‌فرستد.

(ج) تغییر هزینه لینک به گونه ای که x مسیر کمینه جدید به u را اعلام نکند

باید هزینه ای را تغییر دهیم که کمینه $D_x(u)$ ثابت بماند (و حتی المقدور خود بردار x هم تغییر نکند). مثلاً اگر

$$c(x, y) : ۵ \rightarrow ۶,$$

آنگاه مسیر از طریق y به u بدتر می شود:

$$c(x, y) + D_y(u) = ۶ + ۶ = ۱۲ > ۷,$$

و مسیر بهینه x به u همچنان از طریق w با هزینه ۷ باقی می ماند؛ پس x همسایه هایش را از مسیر کمینه جدیدی به u مطلع نمی کند.

۴ سوال ۸

یال ها (طبق شکل) به صورت زیرند:

$$c(x, v) = ۳, c(x, y) = ۶, c(x, w) = ۶, c(x, z) = ۸, c(y, v) = ۸, c(y, t) = ۷, c(y, z) = ۱۲,$$

$$c(v, t) = ۴, c(v, u) = ۳, c(v, w) = ۴, c(w, u) = ۳, c(t, u) = ۲.$$

در جدول دایکسترا، N' مجموعه ی گره های نهایی شده است و $D(n), p(n)$ به ترتیب «کمترین هزینه ی فعلی از x تا n » و «پدر/پیشینه مسیر» است.

$D(u), p(u)$	$D(t), p(t)$	$D(w), p(w)$	$D(v), p(v)$	$D(y), p(y)$	$D(z), p(z)$	N'	Step
$\infty, -$	$\infty, -$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x\}$	۰
$۶, v$	$۷, v$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x, v\}$	۱
$۶, v$	$۷, v$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x, v, y\}$	۲
$۶, v$	$۷, v$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x, v, y, w\}$	۳
$۶, v$	$۷, v$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x, v, y, w, u\}$	۴
$۶, v$	$۷, v$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x, v, y, w, u, t\}$	۵
$۶, v$	$۷, v$	$۶, x$	$۳, x$	$۶, x$	$۸, x$	$\{x, v, y, w, u, t, z\}$	۶

نتیجه ی مسیرهای کمینه از x :

$$D(v) = ۳ : x \rightarrow v, \quad D(y) = ۶ : x \rightarrow y, \quad D(w) = ۶ : x \rightarrow w,$$

$$D(u) = ۶ : x \rightarrow v \rightarrow u, \quad D(t) = ۷ : x \rightarrow v \rightarrow t, \quad D(z) = ۸ : x \rightarrow z.$$

۵ سوال (۱۰)

بله. BGP ذاتاً سیاست محور است و AS ها می توانند با Import/Export Policy تصمیم بگیرند کدام مسیرها را بپذیرند و کدام مسیرها را به دیگران اعلان کنند؛ بنابراین Z می تواند «برای Y ترانزیت باشد» اما «برای X ترانزیت نباشد».

۱. ایده اصلی: کنترل اعلان/پذیرش مسیرها (نه کنترل مستقیم بسته ها)

BGP روی پیشوند مقصد تصمیم می گیرد. پس «ترانزیت ندادن برای X» یعنی: مسیرهای متعلق به X (پیشوندهای X) را یا اصلاً نپذیر یا به هیچ همسایه دیگری صادر نکن تا شبکه های دیگر نتوانند از طریق Z به X برسند.

۲. چطور Z مسیرهای Y را از X تشخیص می دهد؟ (با AS-PATH)

اگر Y مسیرهای خودش را به Z اعلان کند، AS-PATH معمولاً به شکل زیر است:

Routes of Y : [Y]

و اگر (به هر دلیل) مسیرهای X را هم به Z اعلان کند:

Routes of X learned via Y : [Y X]

پس Z می تواند یک فیلتر ورودی (Import Filter) بگذارد که: مسیرهایی را که AS-PATH شامل X است رد کند، اما مسیرهای Y را بپذیرد.

۳. اجرای دقیق سیاست «ترانزیت برای Y ولی نه برای X»

□ قبول کردن مسیرهای Y: پذیرش پیشوندهای Y و (در صورت داشتن همسایه های دیگر) صادر کردن آنها به سایر همسایه ها $\Rightarrow Z$ برای Y ترانزیت می شود.

□ ترانزیت ندادن برای X: مسیرهای X را اصلاً قبول نکند (Import deny) یا حتی اگر برای مصرف داخلی قبول کرد، آنها را به هیچ همسایه دیگری صادر نکند (Export deny).

۴. نکته مهم عملی (قانون رایج Peering)

چون هر دو رابطه X-Y و Y-Z از نوع Peering است، طبق سیاست رایج peering، معمولاً Y اصلاً نباید مسیرهای X را به Z اعلان کند (یعنی Y خودش «سد» می شود). با این حال اگر اعلان انجام شود (مثلاً به خاطر تنظیمات خاص/اشتباه)، Z همچنان با فیلترهای BGP بالا می تواند سیاست را اعمال کند.

BGP به Z اجازه می‌دهد با Import/Export Policy (مثل فیلتر AS-PATH یا Prefix-list) ترانزیت برای Y را فراهم کند ولی برای X ترانزیت فراهم نکند.

۶ سوال ۱۱ - پیاده‌سازی ICMP Pinger و ICMP Traceroute

۱.۶ هدف تمرین و خروجی نهایی

در این سوال، هدف پیاده‌سازی دو ابزار کلاسیک Network Troubleshooting با استفاده از Raw Socket در پایتون بود:

□ **ICMP Pinger**: ارسال ICMP Echo Request و دریافت Echo Reply برای اندازه‌گیری RTT، استخراج TTL از هدر IPv4 و محاسبه Packet Loss، همراه با آمارهای Min/Avg/Max و مدیریت خطاهای ICMP.

□ **ICMP Traceroute** (امتیازی): کشف مسیر گام‌به‌گام تا مقصد با افزایش تدریجی TTL و تحلیل پیام‌های برگشتی مانند ICMP Time Exceeded (Type=11) تا رسیدن به Echo Reply (Type=0).

در پایان، دو فایل اجرایی Pinger.py و Traceroute.py (به‌همراه اسکرین‌شات‌ها و گزارش) تولید شد و برنامه‌ها روی چند مقصد واقعی اجرا شدند تا اثر فاصله جغرافیایی، Filtering و Rate Limiting در ICMP به‌صورت تجربی مشاهده شود.

۲.۶ محیط اجرا، پیش‌نیازها و ملاحظات سیستم‌عامل

□ به دلیل استفاده از Raw Socket، اجرای برنامه در Windows نیازمند Administrator Privileges است و من برنامه‌ها را از داخل PowerShell در حالت Run as Administrator اجرا کردم.

□ در شبکه واقعی، پاسخ‌های ICMP (به‌خصوص Time Exceeded) ممکن است توسط Firewall یا سیاست‌های امنیتی Drop/Filter/Rate-Limit شوند؛ بنابراین Timeout الزاماً نشانه قطع بودن مسیر نیست و یکی از رفتارهای طبیعی شبکه است.

□ وابستگی مسیر به ISP، NAT، سیاست‌های Peering/Transit و حتی وضعیت VPN بسیار مهم است؛ فعال بودن VPN می‌تواند برخی Hop‌ها را «داخل تونل» پنهان کند یا مسیر را به‌کلی تغییر دهد.

۳.۶ پس‌زمینه فنی: ICMP، ساختار بسته و معنای TTL

۱.۳.۶ ساختار ICMP Echo

پیام ICMP Echo از دو بخش تشکیل می‌شود:

□ هدر ICMP (۸ بایت): شامل Type، Code، Checksum، Identifier، Sequence.

□ داده (Payload): در این پروژه یک timestamp (از نوع double) داخل payload قرار داده شد تا RTT به‌صورت دقیق قابل محاسبه باشد.

۲.۳.۶ چک‌سام ICMP (یک توضیح علمی دقیق)

ICMP Checksum برابر است با one's complement مجموع 16-bit کلمات «header + داده». یعنی:

□ داده به صورت 16-bit words جمع می‌شود،

□ carryها به پایین برگردانده می‌شوند (end-around carry)،

□ سپس one's complement گرفته می‌شود.

به همین خاطر هنگام محاسبه چک‌سام باید دقت شود که داده‌ها واقعاً از جنس bytes باشند و ترتیب بایت‌ها Network Byte Order باشد. در نسخه‌های اولیه/کدهای قدیمی مبتنی بر پایتون^۲، این بخش معمولاً نیازمند بازنویسی برای پایتون^۳ است (به‌خاطر تفاوت str/bytes).

۳.۳.۶ نقش TTL

در IPv4، فیلد TTL در هدر IP قرار دارد و هر روتر در مسیر آن را یک واحد کم می‌کند. اگر TTL به صفر برسد:

□ روتر بسته را Drop می‌کند،

□ و یک پیام ICMP Time Exceeded (Type=11) برای فرستنده ارسال می‌کند.

بنابراین:

□ در Ping معمولاً TTL گزارش‌شده مربوط به بسته پاسخ است (و از روی آن به‌تنهایی نمی‌توان تعداد hop را دقیق نتیجه گرفت، چون initial TTL مقصد نامعلوم است).

□ در Traceroute ما عمداً TTL را کنترل می‌کنیم تا hopها آشکار شوند.

۴.۶ بخش اول: ICMP Pinger

۱.۴.۶ روش پیاده سازی (گام به گام)

برای پیاده سازی Ping، دقیقاً این مراحل انجام شد:

۱. ساخت ICMP Echo Request با Type=8 و Code=0.
۲. قرار دادن timestamp داخل Payload برای محاسبه RTT.
۳. محاسبه Checksum روی «Header + Payload» و قرار دادن آن در هدر.
۴. ارسال بسته از طریق Raw Socket.
۵. دریافت بسته پاسخ، جداسازی IP Header و سپس خواندن ICMP Header.
۶. تطبیق Identifier و Sequence برای اطمینان از اینکه پاسخ مربوط به درخواست خودمان است.
۷. استخراج TTL از هدر IPv4 و محاسبه : $\text{Min/Avg/MaxRTT} = t_{recv} - t_{sent}$ ، نرخ Loss و در صورت نیاز شاخص های تغییرپذیری زمان (Jitter).

۲.۴.۶ نکته فنی مهم: جدا کردن درست IP header

در بسیاری از پیاده سازی های ساده فرض می شود هدر IP همیشه ۲۰ بایت است. این فرض معمولاً درست است اما از نظر علمی بهتر است توجه کنیم که طول هدر IP می تواند به خاطر Options تغییر کند (اگرچه در اینترنت عمومی کمتر رایج است). بنابراین اگر بخواهیم robust تر باشیم:

۱. IHL از نیم بایت پایین بایت اول هدر به دست می آید،

۲. و طول واقعی هدر برابر $4 * \text{IHL}$ خواهد بود.

(در کد من ساده سازی ۲۰ بایت هم جواب داده، اما این نکته برای تحلیل علمی مسیر مهم است.)

۳.۴.۶ بخشی از کد Pinger: ساخت بسته ICMP و قرار دادن timestamp

```
1 # Header: type(8), code(8), checksum(16), id(16), sequence(16)
2 header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, 0, ID, seq)
```

3


```

4 # Payload: timestamp
5 data = struct.pack("d", time.time())
6
7 myChecksum = checksum(header + data)
8 myChecksum = htons(myChecksum)
9
10 header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, myChecksum, ID, seq)
11 packet = header + data
12
13 mySocket.sendto(packet, (destAddr, 1))

```

۴.۴.۶ بخشی از کد Pinger: دریافت پاسخ، استخراج TTL و محاسبه RTT

```

1 recPacket, addr = mySocket.recvfrom(1024)
2
3 icmpHeader = recPacket[20:28]
4 icmpType, code, checksum, packetID, sequence = struct.unpack("bbHHh", icmpHeader)
5
6 if icmpType == 0 and packetID == ID:
7     bytesInDouble = struct.calcsize("d")
8     timeSent = struct.unpack("d", recPacket[28:28 + bytesInDouble])[0]
9
10     # TTL in IPv4 header (byte offset 8)
11     ttl = struct.unpack("B", recPacket[8:9])[0]
12
13     rtt = (timeReceived - timeSent) * 1000
14     return (rtt, ttl, len(recPacket))

```

۵.۴.۶ مدیریت خطاهای ICMP (اهمیت علمی)

علاوه بر Echo Reply (Type=0)، در عمل ممکن است پاسخ‌هایی مثل موارد زیر دریافت شود:

□ Destination Unreachable (Type=3): مقصد یا یک شبکه میانی اعلام می کند دسترسی ممکن نیست.

□ Time Exceeded (Type=11): معمولاً در سناریوهای traceroute یا در صورت TTL کم رخ می دهد.

داشتن این مدیریت خطا باعث می شود برنامه «تفاوت» بین Timeout ساده، Filtering، و واقعاً Unreachable بودن مسیر را بهتر نشان دهد.

۶.۴.۶ روش محاسبه آمارها

برای هر مقصد، چند بسته ارسال شد و سپس:

□ Packet Loss با نسبت (Sent-Received)/Sent گزارش شد.

□ از بین RTT های موفق، Min/Avg/Max محاسبه شد.

□ از نظر علمی، Avg RTT یک شاخص کلی است ولی Jitter (تغییرپذیری RTT) هم مهم است؛ چون jitter معمولاً ناشی از queueing delay، تغییر مسیر لحظه ای (route flap) یا load balancing در مسیر است.

۷.۴.۶ اسکرین شات های خروجی Ping

```
PS C:\Users\a\Desktop\CN> python Pinger.py

[Testing: Localhost]
Pinging 127.0.0.1 using Python:

Reply from 127.0.0.1: bytes=36 time=0.00ms TTL=128
Reply from 127.0.0.1: bytes=36 time=0.00ms TTL=128
Reply from 127.0.0.1: bytes=36 time=0.51ms TTL=128
Reply from 127.0.0.1: bytes=36 time=0.00ms TTL=128

--- 127.0.0.1 ping statistics ---
    Packets: Sent = 4, Received = 4, Lost = 0 (0.0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0.00ms, Maximum = 0.51ms, Average = 0.13ms
```

شکل ۱: خروجی Ping برای 127.0.0.1 (تست محلی)

```
[Testing: Asia (Tokyo, Japan)]
Pinging 210.152.243.234 using Python:

Reply from 210.152.243.234: bytes=36 time=0.51ms TTL=64
Reply from 210.152.243.234: bytes=36 time=1.11ms TTL=64
Reply from 210.152.243.234: bytes=36 time=2.20ms TTL=64
Reply from 210.152.243.234: bytes=36 time=1.47ms TTL=64

--- www.u-tokyo.ac.jp ping statistics ---
    Packets: Sent = 4, Received = 4, Lost = 0 (0.0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0.51ms, Maximum = 2.20ms, Average = 1.32ms
```

شکل ۲: خروجی Ping برای آسیا: www.u-tokyo.ac.jp

```
[Testing: Europe (London, UK)]
Pinging 23.185.0.3 using Python:

Reply from 23.185.0.3: bytes=36 time=0.60ms TTL=64
Reply from 23.185.0.3: bytes=36 time=1.43ms TTL=64
Reply from 23.185.0.3: bytes=36 time=1.04ms TTL=64
Reply from 23.185.0.3: bytes=36 time=0.88ms TTL=64

--- www.cam.ac.uk ping statistics ---
    Packets: Sent = 4, Received = 4, Lost = 0 (0.0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0.60ms, Maximum = 1.43ms, Average = 0.99ms
```

شکل ۳: خروجی Ping برای اروپا: www.cam.ac.uk

```
[Testing: North America (California, USA)]
Pinging 151.101.194.133 using Python:

Reply from 151.101.194.133: bytes=36 time=1.06ms TTL=64
Reply from 151.101.194.133: bytes=36 time=0.88ms TTL=64
Reply from 151.101.194.133: bytes=36 time=1.42ms TTL=64

--- www.stanford.edu ping statistics ---
    Packets: Sent = 4, Received = 4, Lost = 0 (0.0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0.88ms, Maximum = 1.42ms, Average = 1.15ms
```

شکل ۴: خروجی Ping برای آمریکای شمالی: www.stanford.edu

```
[Testing: Oceania (Melbourne, Australia)]
Pinging 2.58.104.10 using Python:

Reply from 2.58.104.10: bytes=36 time=4.82ms TTL=64
Reply from 2.58.104.10: bytes=36 time=0.97ms TTL=64
Reply from 2.58.104.10: bytes=36 time=0.75ms TTL=64
Reply from 2.58.104.10: bytes=36 time=1.12ms TTL=64

--- www.unimelb.edu.au ping statistics ---
    Packets: Sent = 4, Received = 4, Lost = 0 (0.0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0.75ms, Maximum = 4.82ms, Average = 1.91ms
```

شکل ۵: خروجی Ping برای اقیانوسیه: www.unimelb.edu.au


```
[Testing: Africa (Cape Town, SA)]
Pinging 137.158.159.192 using Python:

Reply from 137.158.159.192: bytes=36 time=1.29ms TTL=64
Reply from 137.158.159.192: bytes=36 time=1.50ms TTL=64
Reply from 137.158.159.192: bytes=36 time=0.83ms TTL=64
Reply from 137.158.159.192: bytes=36 time=1.17ms TTL=64

--- www.uct.ac.za ping statistics ---
    Packets: Sent = 4, Received = 4, Lost = 0 (0.0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0.83ms, Maximum = 1.50ms, Average = 1.20ms
```

شکل ۶: خروجی Ping برای آفریقا: www.uct.ac.za

۸.۴.۶ خلاصه عددی نتایج Ping (بر اساس اجرای واقعی من)

RTT Max	RTT Avg	RTT Min	Loss	IP	Target
0.51 ms	0.13 ms	0.00 ms	0%	127.0.0.1	Localhost
2.20 ms	1.32 ms	0.51 ms	0%	210.152.243.234	Asia (Tokyo)
1.43 ms	0.99 ms	0.60 ms	0%	23.185.0.3	Europe (Cambridge)
1.42 ms	1.15 ms	0.88 ms	0%	151.101.194.133	North America (Stanford)
4.82 ms	1.91 ms	0.75 ms	0%	2.58.104.10	Oceania (Melbourne)
1.50 ms	1.20 ms	0.83 ms	0%	137.158.159.192	Africa (Cape Town)

جدول ۱: خلاصه نتایج Ping بر اساس اجرای واقعی من

۹.۴.۶ تحلیل علمی نتایج Ping (تفسیر کامل تر)

□ تست محلی Localhost: RTT بسیار کوچک است چون مسیر عملاً از کارت شبکه خارج نمی شود و بسته داخل Kernel/Network Stack پردازش می شود. در این حالت سهم غالب تأخیر مربوط به context switch و پردازش نرم افزاری است نه انتقال فیزیکی.

□ برداشت درست از TTL: Ping در خروجی TTL چاپ شده مربوط به بسته پاسخ است. از آنجا که مقدار اولیه TTL در سیستم مقصد می تواند متفاوت باشد (مثلاً ۶۴ یا ۱۲۸ یا ۲۵۵)، صرف دیدن یک TTL خاص،

معادل «تعداد hop» نیست. برای تخمین تقریبی hop معمولاً باید حدس بزنیم مقصد با چه initial TTL پاسخ می‌دهد، اما این حدس همیشه قطعی نیست.

□ چرا ممکن است RTT «کمتر از انتظار» دیده شود؟ در اینترنت واقعی، چند عامل کلیدی باعث می‌شود نام دامنه‌ای که فکر می‌کنیم «دور» است، در عمل به یک سرور نزدیک نگاشت شود:

□ CDN / Edge Caching: بسیاری از وبسایت‌ها پشت CDN هستند و DNS نزدیک‌ترین edge را برمی‌گرداند.

□ Anycast: یک آدرس IP ممکن است از چند نقطه دنیا announce شود و به نزدیک‌ترین PoP هدایت گردد.

□ Policy Routing / Peering: اپراتورها ممکن است مسیرهای متفاوتی برای برخی مقاصد انتخاب کنند.

بنابراین «نام مقصد» لزوماً هم‌معنی «محل فیزیکی سرور» نیست و این نکته در تحلیل علمی خروجی‌ها بسیار مهم است.

□ تحلیل Loss صفر Loss=0% نشان می‌دهد در لحظه اجرا، مسیر پایدار بوده و پاسخ‌های ICMP Echo Reply توسط Firewall مقصد یا مسیر فیلتر نشده است. با این حال در بسیاری از شبکه‌ها، ICMP ممکن است محدود شود و در اجرای مجدد امکان مشاهده loss وجود دارد.

۵.۶ بخش دوم (امتیازی): ICMP Traceroute

۱.۵.۶ ایده اصلی Traceroute (توضیح دقیق و علمی)

ایده Traceroute مبتنی بر این واقعیت است که TTL در هر روتر یک واحد کم می‌شود. پس اگر بسته‌ای با TTL=1 ارسال کنیم:

□ در روتر اول TTL صفر می‌شود،

□ روتر اول بسته را drop می‌کند،

□ و ICMP Time Exceeded (Type=11) برمی‌گرداند.

حال اگر TTL را ۲ کنیم، روتر دوم پاسخ می‌دهد، و به همین ترتیب می‌توان hopها را یکی‌یکی کشف کرد تا در نهایت مقصد با Echo Reply (Type=0) پاسخ دهد.

۲.۵.۶ روش پیاده سازی (گامها)

برای هر TTL از ۱ تا ۳۰:

۱. ساخت ICMP Echo Request مثل ping

۲. تنظیم TTL روی سوکت با `setsockopt(IP_TTL)`

۳. ارسال بسته و ثبت زمان ارسال

۴. دریافت پاسخ و تحلیل نوع پیام:

□ `Type=11`: یک hop جدید کشف شده است.

□ `Type=0`: به مقصد رسیده ایم و `traceroute` تمام می شود.

□ `Timeout`: پاسخ ICMP از آن hop دریافت نشده است (به دلایل طبیعی مثل `Filtering/Rate-Limit`).

۳.۵.۶ چالش های عملی مهم (تجربه واقعی اجرا)

در عمل، چند چالش رایج وجود دارد:

□ سازگاری پایتون ۳ و داده های باینری: در کدهای قدیمی پایتون ۲، بخش هایی مثل `checksum` یا ارسال داده ممکن است با `string` نوشته شده باشد؛ در پایتون ۳ باید همه چیز دقیقاً `bytes` باشد. بازنویسی `checksum` و بسته بندی با `struct.pack` معمولاً ضروری است.

□ فایروال ویندوز و پیام های `Time Exceeded`: در برخی تنظیمات، فایروال می تواند دریافت `Type=11` را محدود کند و خروجی `traceroute` فقط * * * شود. اجرای برنامه با دسترسی `admin` و (در برخی موارد) تنظیم مناسب فایروال باعث می شود hopها درست نمایش داده شوند.

□ `Reverse DNS`: برای خواناتر شدن خروجی، می توان IP هر hop را با `reverse DNS` به نام دامنه تبدیل کرد؛ اما این مرحله خودش ممکن است کند باشد یا در برخی hopها جواب ندهد (پس باید خطا مدیریت شود).

۴.۵.۶ بخشی از کد `Traceroute`: تنظیم TTL و تشخیص نوع پیام ICMP

```
1 mySocket = socket(AF_INET, SOCK_RAW, icmp_proto)
2 mySocket.setsockopt(IPPROTO_IP, IP_TTL, ttl)
3 mySocket.settimeout(TIMEOUT)
```

```

4
5 packet = build_packet()
6 t_send = time.time()
7 mySocket.sendto(packet, (destAddr, 0))
8
9 recvPacket, addr = mySocket.recvfrom(4096)
10 icmpHeader = recvPacket[20:28]
11 types, code, checksum, packetID, sequence = struct.unpack("bbHHh", icmpHeader)
12
13 if types == 11:
14     # Time Exceeded: hop found
15 elif types == 0:
16     # Echo Reply: destination reached

```

۵.۵.۶ چرا Timeout در traceroute طبیعی است؟ (تحلیل علمی)

Timeout در traceroute معمولاً یکی از دلایل زیر را دارد:

- ICMP Rate Limiting: روتر برای جلوگیری از فشار پردازشی، پاسخ ICMP را محدود می کند.
 - Firewall Policy / ICMP Filtering: بعضی شبکه ها برای پنهان کردن توپولوژی یا اهداف امنیتی، پیام های Time Exceeded را نمی فرستند.
 - Asymmetric Routing: مسیر رفت و برگشت ممکن است متفاوت باشد و پیام برگشتی از مسیر دیگری بیاید که به ما نرسد.
 - Load Balancing: اگر بین چند مسیر تقسیم بار باشد، ممکن است در هر probe hop متفاوتی دیده شود.
- بنابراین وجود ستاره ها در میانه مسیر، در بسیاری از شبکه های Backbone و مخصوصاً شبکه های ابری، انتظار می رود.

۶.۵.۶ اسکرین شات های خروجی Traceroute

```

Traceroute to google.com over a maximum of 30 hops:

 1 rtt=2 ms 172.30.96.1 [hostname not found]
 2 rtt=2 ms 172.16.81.1 [hostname not found]
 3 rtt=16 ms 192.168.40.1 [hostname not found]
 4 rtt=56 ms 192.168.32.3 [hostname not found]
 5 rtt=4 ms 172.17.49.17 [hostname not found]
 6 rtt=3 ms 172.17.49.98 [hostname not found]
 7 rtt=5 ms 192.168.61.209 [hostname not found]
 8 rtt=4 ms 194.225.150.20 [nia-sw-150-20.ipm.edge-2.iranet.ir]
 9 rtt=9 ms 194.225.151.5 [1-2-v102.lct-ro-151-5.ipm.edge-2.iranet.ir]
10 rtt=74 ms 213.198.52.64 [xe-0-4-2-1.a03.frnkge07.de.bb.gin.ntt.net]
11 * * * Request timed out.
11 * * * Request timed out.
12 rtt=75 ms 129.250.2.5 [ae-1.a02.frnkge13.de.bb.gin.ntt.net]
13 rtt=76 ms 129.250.8.203 [ae-0.tata-communications.frnkge13.de.bb.gin.ntt.net]
14 rtt=76 ms 142.250.167.180 [hostname not found]
15 rtt=93 ms 192.178.108.183 [hostname not found]
16 rtt=78 ms 142.250.214.193 [hostname not found]
17 rtt=77 ms 142.250.186.110 [fra24s06-in-f14.1e100.net]

```

شکل ۷: خروجی Traceroute برای google.com

```

Traceroute to www.cam.ac.uk (23.185.0.3), 30 hops max:
 1 2ms 2ms 192.168.1.1
 2 73ms 14ms 100.91.0.1
 3 31ms 13ms 192.168.200.245
 4 15ms 19ms 192.168.200.246
 5 27ms 16ms 10.22.29.61
 6 16ms 15ms 10.22.29.62
 7 20ms 28ms 10.22.29.65
 8 * * Request timed out.
 9 * * Request timed out.
10 * * Request timed out.
11 * * Request timed out.
12 * * Request timed out.
13 85ms 83ms 23.185.0.3

```

شکل ۸: خروجی Traceroute برای www.cam.ac.uk

```

Traceroute to www.baidu.com (103.235.46.102), 30 hops max:
 1  2ms  3ms  192.168.1.1
 2 1011ms 1089ms 100.91.0.1
 3 954ms 833ms 192.168.200.245
Traceroute to www.baidu.com (103.235.46.102), 30 hops max:
 1  2ms  3ms  192.168.1.1
 2 1011ms 1089ms 100.91.0.1
 3 954ms 833ms 192.168.200.245
 4 751ms 779ms 192.168.200.246
 2 1011ms 1089ms 100.91.0.1
 3 954ms 833ms 192.168.200.245
 4 751ms 779ms 192.168.200.246
 4 751ms 779ms 192.168.200.246
 5 1163ms 1094ms 10.22.29.61
 6 17ms 15ms 10.22.29.62
 7 37ms 30ms 10.22.29.65
 8 23ms 17ms 10.21.249.202
 9 * * Request timed out.
10 * * Request timed out.
11 93ms 92ms peering-85.132.90.129.az-ix.net [85.132.90.129]
12 * * Request timed out.
13 102ms 101ms ix-ge-100-0-0-67.qhar1.stk-stockholm.as6453.net [80.231.30.66]
14 336ms 338ms if-bundle-12-2.qcore1.stk-stockholm.as6453.net [195.219.36.19]
15 340ms * if-bundle-19-2.qcore1.fnm-frankfurt.as6453.net [195.219.68.80]
16 * * Request timed out.
17 * 339ms if-bundle-14-2.qcore2.emrs2-marseille.as6453.net [195.219.187.6]
18 * * Request timed out.
19 * * Request timed out.
20 * * Request timed out.
21 336ms 337ms if-bundle-7-2.qcore1.h81-hongkong.as6453.net [180.87.146.206]
22 337ms 336ms 180.87.162.46
23 * * Request timed out.
24 * * Request timed out.
25 * * Request timed out.
26 339ms 339ms 103.235.46.102

```

شکل ۹: خروجی Traceroute برای www.baidu.com

```

Traceroute to www.sydney.edu.au (18.65.39.3), 30 hops max:
 1  3ms  2ms  192.168.1.1
 2 53ms 28ms 100.91.0.1
 3 18ms 21ms 192.168.200.245
 4 19ms 17ms 192.168.200.246
 5 21ms 19ms 10.22.29.61
 6 16ms 15ms 10.22.29.62
 7 17ms 22ms 10.22.29.65
 8 18ms * 10.21.249.202
 9 * * Request timed out.
10 * * Request timed out.
11 90ms 91ms 213.144.184.56
12 90ms 104ms 195.22.211.33
13 * * Request timed out.
14 104ms 103ms be3343.ccr41.ams03.atlas.cogentco.com [154.54.62.142]
15 * * Request timed out.
16 * * Request timed out.
17 * * Request timed out.
18 * * Request timed out.
19 * * Request timed out.
20 * * Request timed out.
21 * * Request timed out.
22 * * Request timed out.
23 * * Request timed out.
24 * * Request timed out.
25 * * Request timed out.
26 * * Request timed out.
27 173ms 177ms server-18-165-220-44.bah53.r.cloudfront.net [18.165.220.44]

```

شکل ۱۰: خروجی Traceroute برای www.sydney.edu.au

۷.۵.۶ بازنویسی متنی خروجی‌ها (بدون بلاک کامند) + تحلیل عمیق‌تر مسیرها

۱) مسیر google.com مشاهده کلیدی: چند hop اول شامل Private IP هاست (مثل 172.16.x.x و 192.168.x.x) که نشان می‌دهد بخش ابتدایی مسیر داخل شبکه محلی/پراتور و پشت NAT است. سپس از hop های میانی، دامنه‌هایی مثل iranet.ir و بعد ntt.net دیده می‌شود که نشانه ورود به Transit/Backbone بین‌المللی است. تحلیل Timeout: Timeout در یک hop میانی می‌تواند ناشی از Rate-Limit باشد؛ مهم این است که مسیر بعد از آن ادامه پیدا کرده و به مقصد رسیده است؛ پس Timeout به معنی قطع شدن مسیر نیست.

۲) مسیر www.cam.ac.uk مشاهده کلیدی: وجود چند Timeout پشت‌سرهم ولی رسیدن نهایی به مقصد نشان می‌دهد برخی روترهای میانی پاسخ Time Exceeded را ارسال نمی‌کنند (یا فیلتر شده) اما مسیر داده‌ای برقرار

است.

تحلیل اختلاف زمان‌ها: اختلاف‌های زیاد بین دو اندازه‌گیری در یک hop (مثلاً 73ms و 14ms) معمولاً نشانه queueing jitter، تغییر لحظه‌ای مسیر یا تقسیم بار است.

۳) مسیر www.baidu.com مشاهده کلیدی: در خروجی چین، نام‌هایی شامل شهر/نقاط Frankfurt, Stockholm, Marseille و سپس Hongkong دیده می‌شود که یک «سفر بین‌قاره‌ای» را نشان می‌دهد: عبور از مراکز تبادل و backbone اروپا و سپس ورود به آسیای شرقی. تحلیل RTT‌های بزرگ: رسیدن RTT به حدود چندصد میلی‌ثانیه با مسیر طولانی و عبور از لینک‌های بین‌قاره‌ای (به‌خصوص کابل‌های زیردریایی) سازگار است. همچنین وجود Timeout‌های متعدد در backbone طبیعی است.

۴) مسیر www.sydney.edu.au و نقش CDN مشاهده کلیدی: در انتهای مسیر، دامنه‌هایی مرتبط با cloudfront.net دیده می‌شود؛ این یعنی مقصد پشت Amazon CloudFront (یک CDN) قرار دارد و احتمالاً درخواست به نزدیک‌ترین Edge Location هدایت شده است (نه لزوماً سرور فیزیکی داخل سیدنی). تحلیل Timeout‌های طولانی در میانه: در شبکه‌های ابری و دیتاسنترها، پاسخ به traceroute/ICMP می‌تواند عمداً فیلتر شود تا توپولوژی داخلی پنهان بماند؛ بنابراین زیاد بودن ستاره‌ها لزوماً نشانه مشکل نیست.

۸.۵.۶ اعتبارسنجی با ابزار سیستم‌عامل (tracert) و دلیل اختلاف‌ها

برای sanity check از Windows tracert هم استفاده شد. نکته مهم این است که:

□ ابزارهای سیستم‌عامل نیز ممکن است در hop‌های ابتدایی یا میانی Timeout نشان دهند (بسته به سیاست شبکه).

□ اختلاف بین traceroute سفارشی و tracert می‌تواند ناشی از rate limiting، تفاوت تعداد probe‌ها، DNS، و حتی load balancing باشد.

پس وجود رفتار مشابه (timeouts در برخی hop‌ها ولی ادامه مسیر) به ما می‌گوید مشکل از «کد» نیست، بلکه ویژگی محیط واقعی شبکه است.

۶.۶ جمع‌بندی نهایی و دستاوردهای علمی

در این سوال، من:

□ با Raw Socket کار کردم و بسته‌های ICMP را در سطح باینری ساختم و checksum را محاسبه کردم.

- در Pinger، RTT را از timestamp داخل payload محاسبه کردم، TTL را از هدر IPv4 استخراج کردم و آمارهای Loss/Min/Avg/Max را گزارش دادم.
- در Traceroute، با کنترل TTL و تحلیل پیام‌های ICMP Type 11/0 مسیر hop-by-hop را تا حد ممکن استخراج کردم و timeoutهای طبیعی شبکه را به صورت علمی تفسیر کردم.
- به صورت تجربی دیدم که سیاست‌های امنیتی (مثل فایروال ویندوز یا filtering در backbone/Cloud) می‌تواند روی مشاهده hopها اثر بگذارد و این موضوع جزء واقعیت‌های عملی Network Measurement است.

۷.۶ موارد تحویلی

- Pinger.py
- Traceroute.py
- گزارش PDF (همین فایل)
- پوشه تصاویر اسکرین‌شات‌ها مطابق مسیرهای استفاده‌شده در گزارش

مراجع